Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica

SOSP 2023

Presenter: Seonghun Jung, Changyong Eom

KAIST

Contents

- **Introduction**
  - Background
  - Motivation
- **Approach**
  - Key Idea
  - Design & Implementation
- **Evaluation**
  - Methodology
  - Results
- **Conclusion**
  - Takeaways
  - Discussion

LLM Serving is Expensive

- High LLM serving cost makes **throughput a key system objective**
  - An LLM exchange can cost ~10× more than a standard keyword search (Hennessy, Reuters 2023).
  - Higher **throughput** of system reduces the cost per request.

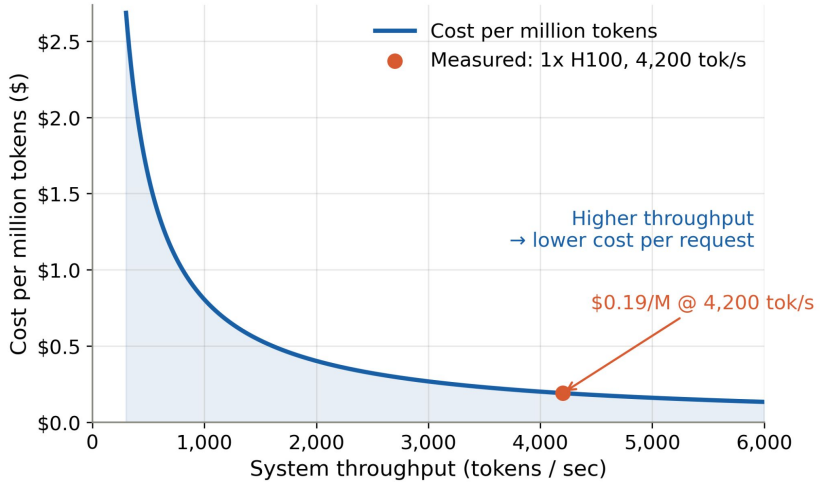


Fig 1. Cost per token vs. throughput (1× H100, $2.90/hr; Spheron, 2026, Llama 4 Scout 17B).

LLM Inference Workflow

- Autoregressive inference has **two phases** and **reuses the KV cache.**
  - **Prefill**: Use parallel computation → High GPU utilization.
  - **Decode**: Generates tokens one by one with GEMV → Low GPU utilization.

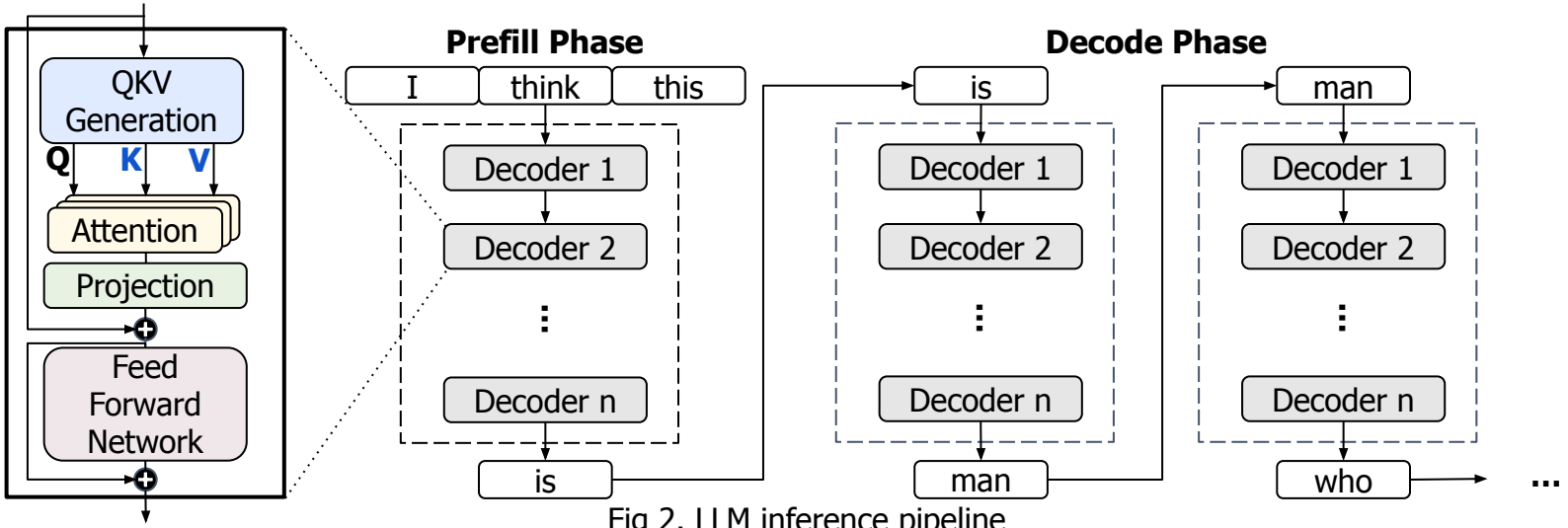


Fig 2. LLM inference pipeline

Maximizing Throughput in LLM Inference

- Overcoming low GPU utilization through batching
  - Problem: Sequential token generation → Low GPU utilization → **Low throughput & High cost**
  - Solution: **Batching multiple requests** → Enable parallel computation & Boost throughput



Fig 3. Fine-grained batching mechanism

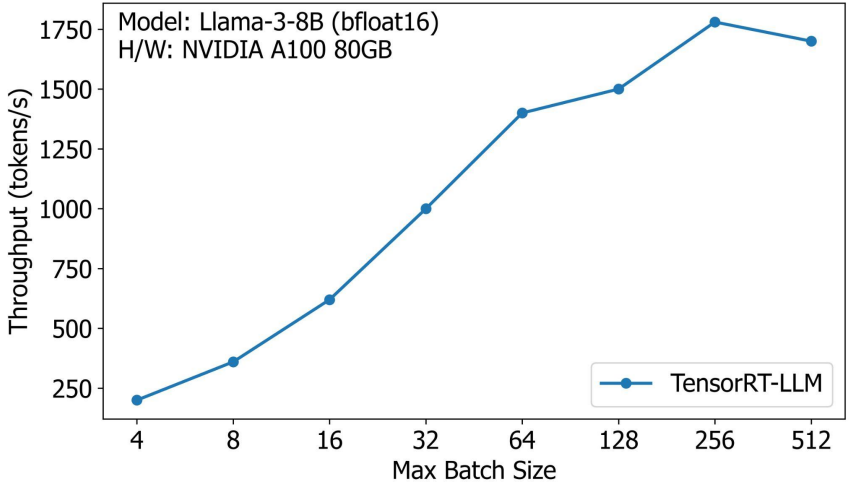


Fig 4. Throughput vs. Batch Size

LLM serving is Memory-Bound

- Batching boosts throughput, **but GPU memory caps the batch size**
  - Bottleneck: Batch size is limited by KV cache space, not compute.

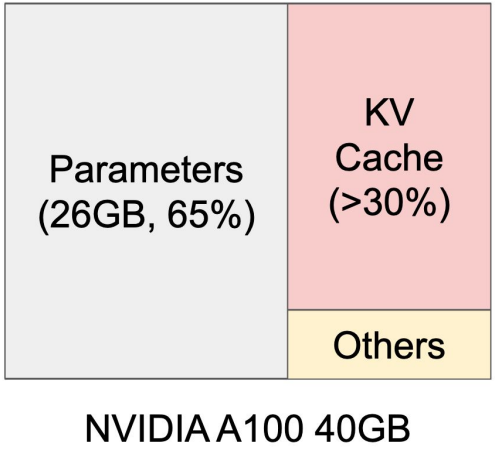


Fig 5. 13B LLM memory layout on A100

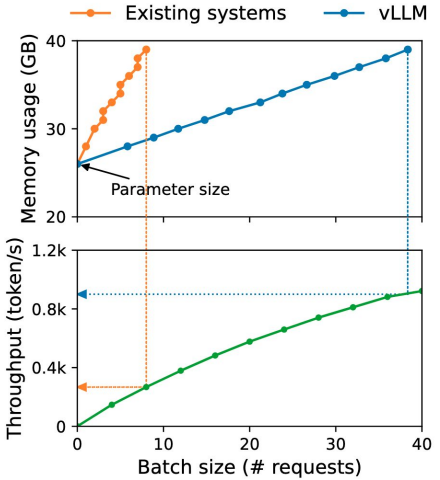


Fig 6. Memory usage and throughput under varying batch sizes

Challenges in KV Cache Management

1) Large and Unpredictable Memory Footprint
   - Large: ~800 KB per token → up to 1.6 GB per request (13B model)
   - Unpredictable: Output length unknown a priori → cannot pre-plan memory

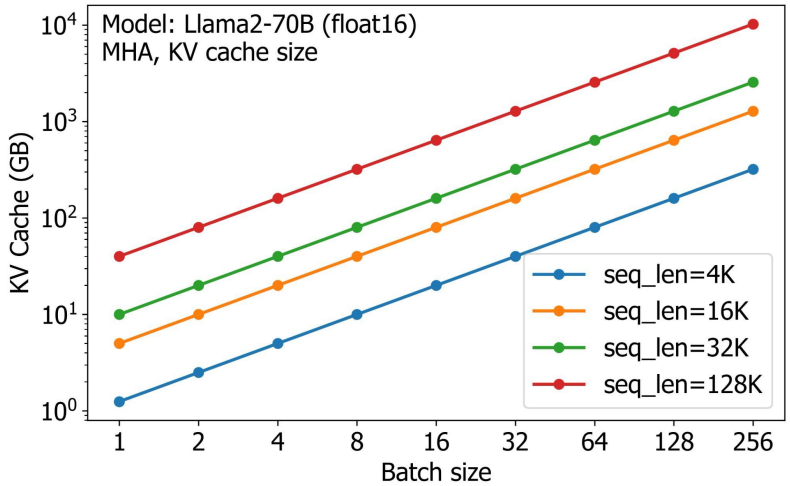


Fig 7. KV cache scaling with batch size and sequence length

Challenges in KV Cache Management

2) Complex Decoding Scenarios

- Sharing opportunities vary by algorithm → **hard to manage uniformly**
- e.g: Parallel sampling shares the prompt only(~12%);
  Beam search shares more(~55%), but the **pattern keeps changing.**

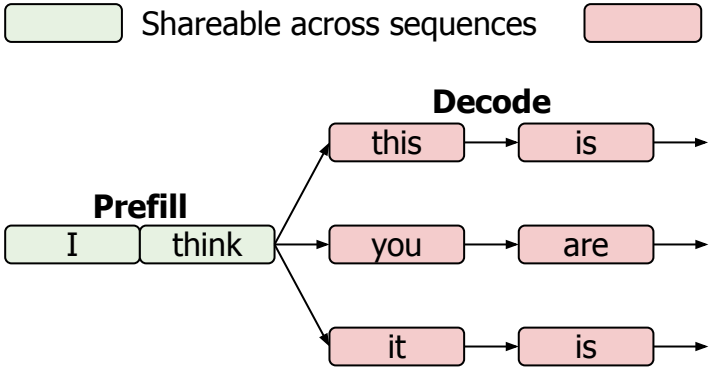


Fig 8. Parallel sampling

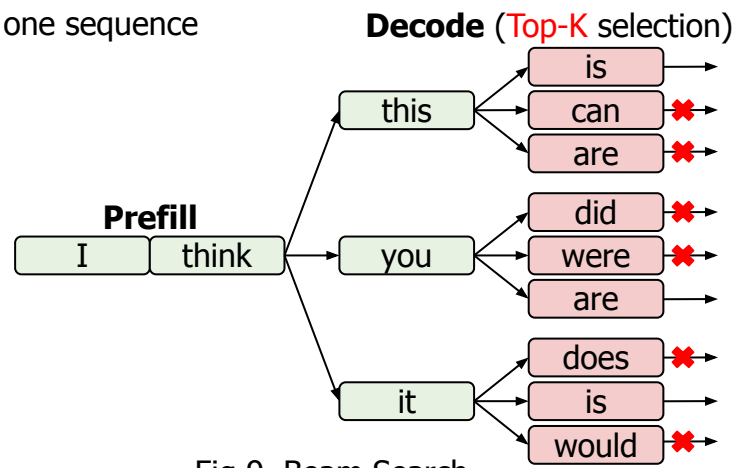


Fig 9. Beam Search

Limitations of Previous Memory Management

- Prior systems store the KV cache contiguously, **pre-allocated to the max length**
  - Pure waste: internal + external fragmentation (never used)
  - Partial waste: reserved space occupied until the request ends

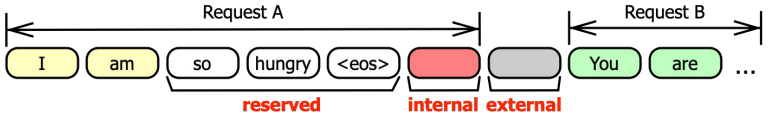


Fig 10. Reserved, internal, external fragmentation

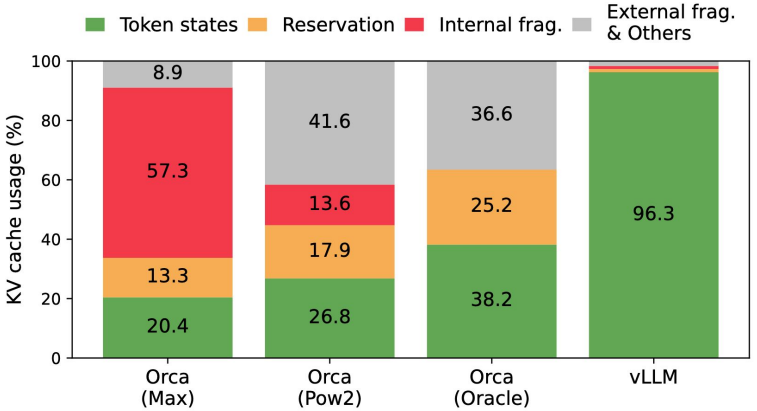


Fig 11. Memory waste breakdown across serving systems

Limitations of Previous Memory Management

- Prior systems store the KV cache contiguously, **pre-allocated to the max length**
  - Pure waste: internal + external fragmentation (never used)
  - Partial waste: reserved space occupied until the request ends

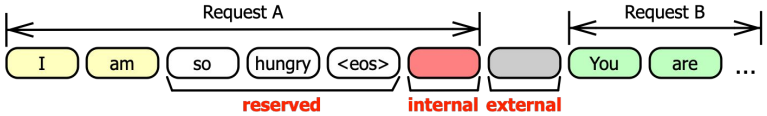


Fig 10. Reserved, internal, external fragmentation

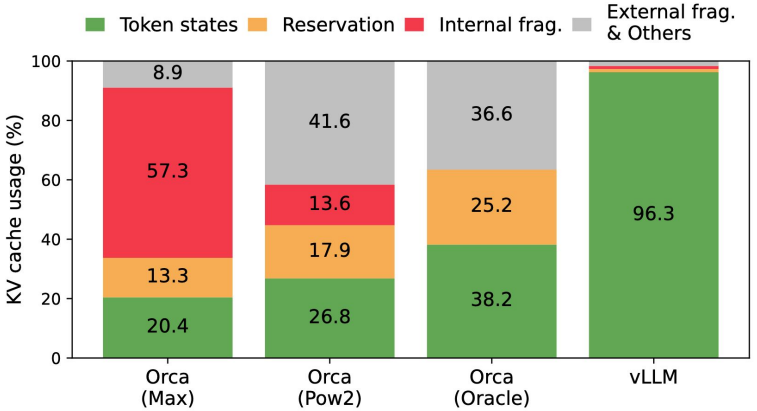


Fig 11. Memory waste breakdown across serving systems

Limitations of Previous Memory Management

- Prior systems store the KV cache contiguously, **pre-allocated to the max length**
  - Pure waste: internal + external fragmentation (never used)

1. **Eliminate memory fragmentation**
2. **Enable flexible KV cache sharing**



Fig 10. Reserved, internal, external fragmentation

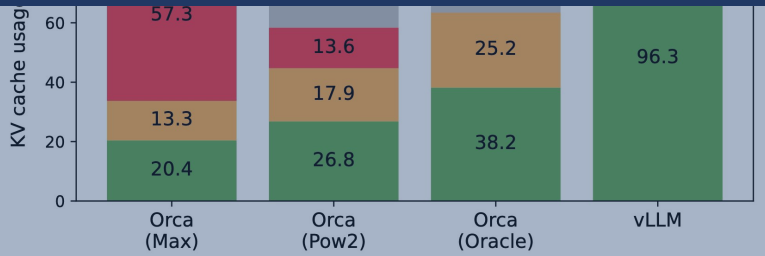


Fig 11. Memory waste breakdown across serving systems

KV Cache as Paged Memory

- Apply OS virtual memory & paging to the KV cache → PagedAttention
  - Split into **fixed-size blocks,** stored non-contiguously (block = page, request = process)
  - **Less fragmentation** (Goal 1) + **blocks shareable across sequences** (Goal 2)

Request prompt: "Four score and seven years ago out"

Output: "fathers" → "brought" → …

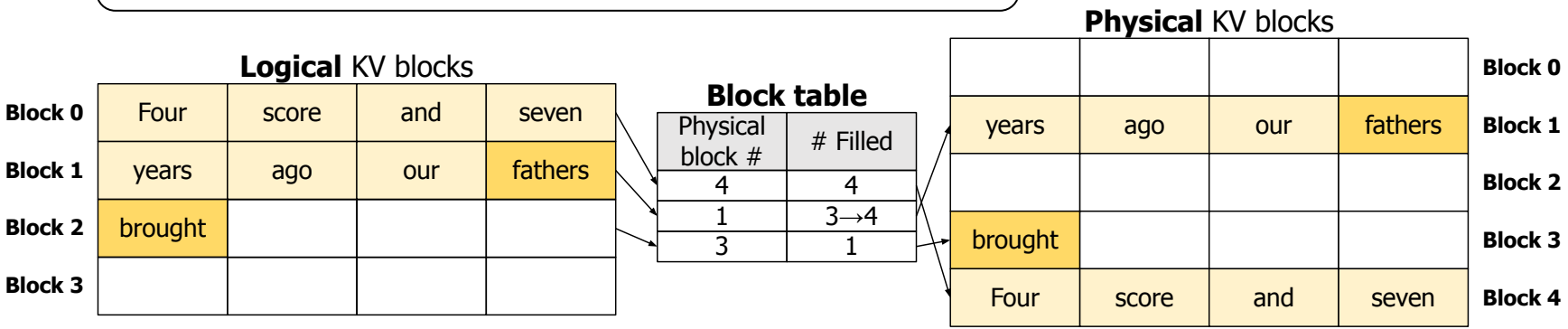
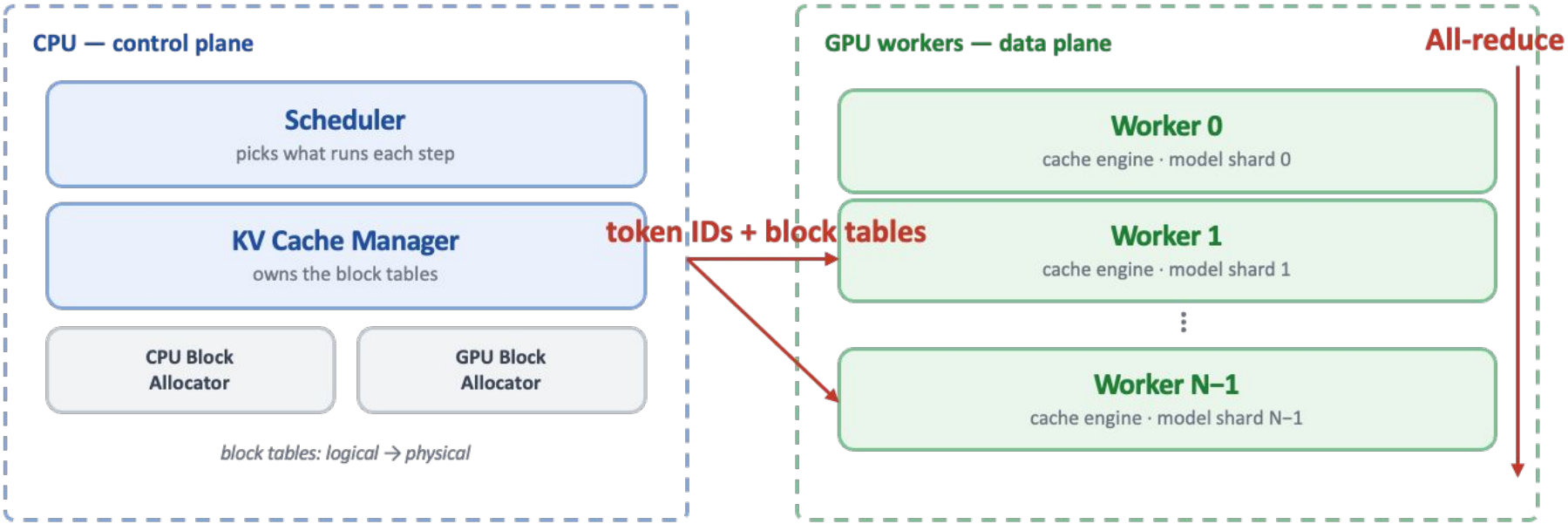


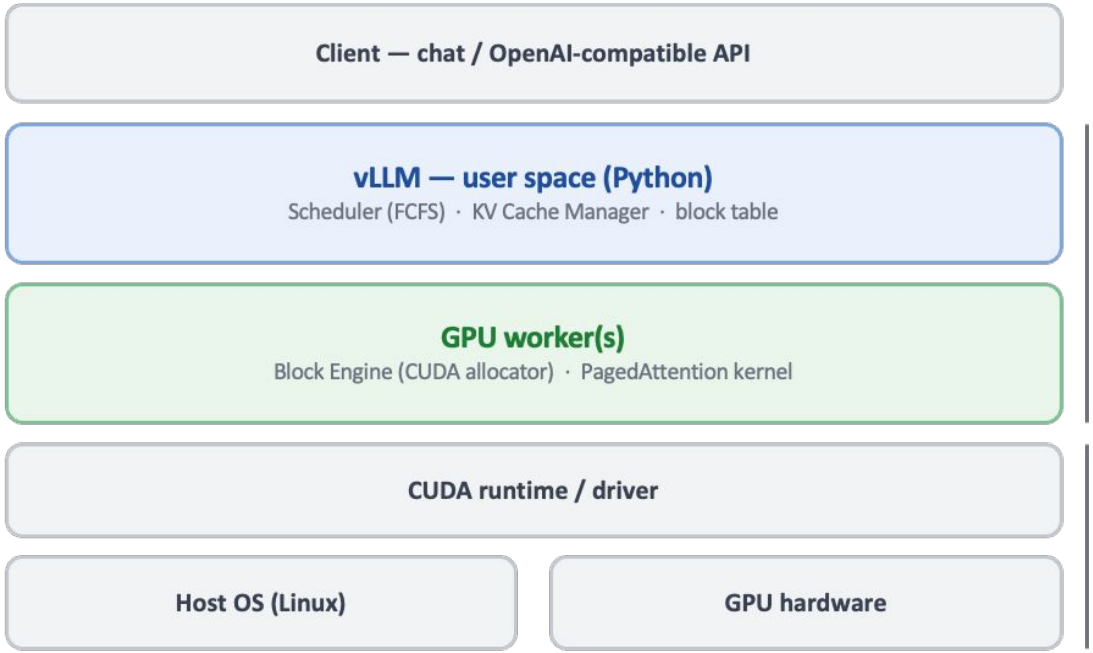
Fig 12. Block table translation with PagedAttention

How vLLM Works



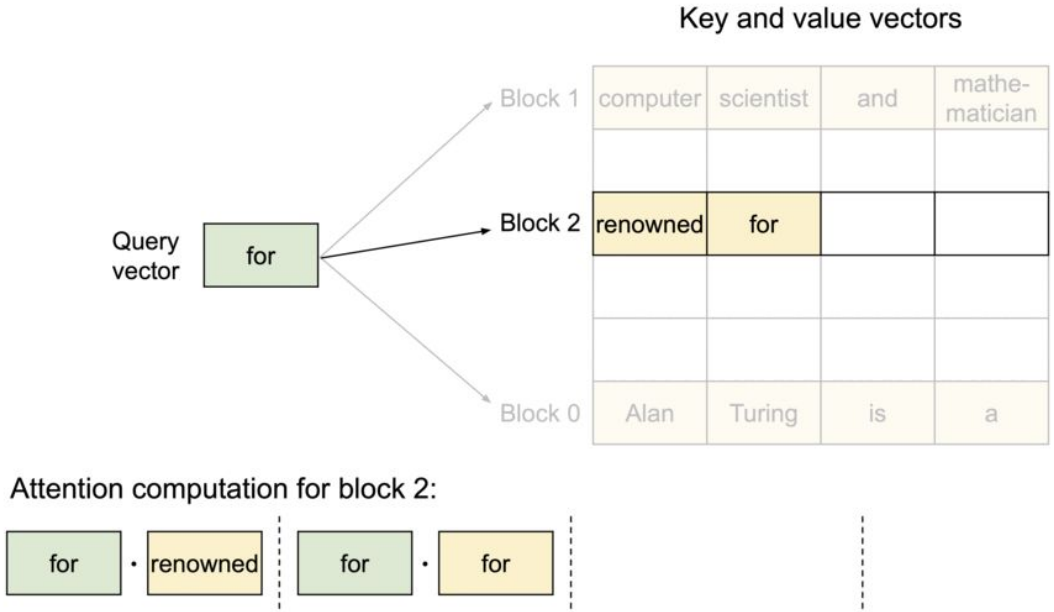
Memory management stays centralized on the CPU; the N GPU workers sync via all-reduce.

vLLM as a User-Space Memory Subsystem



vLLM is not a new OS — it's a user-space virtual-memory subsystem for the GPU's KV cache.

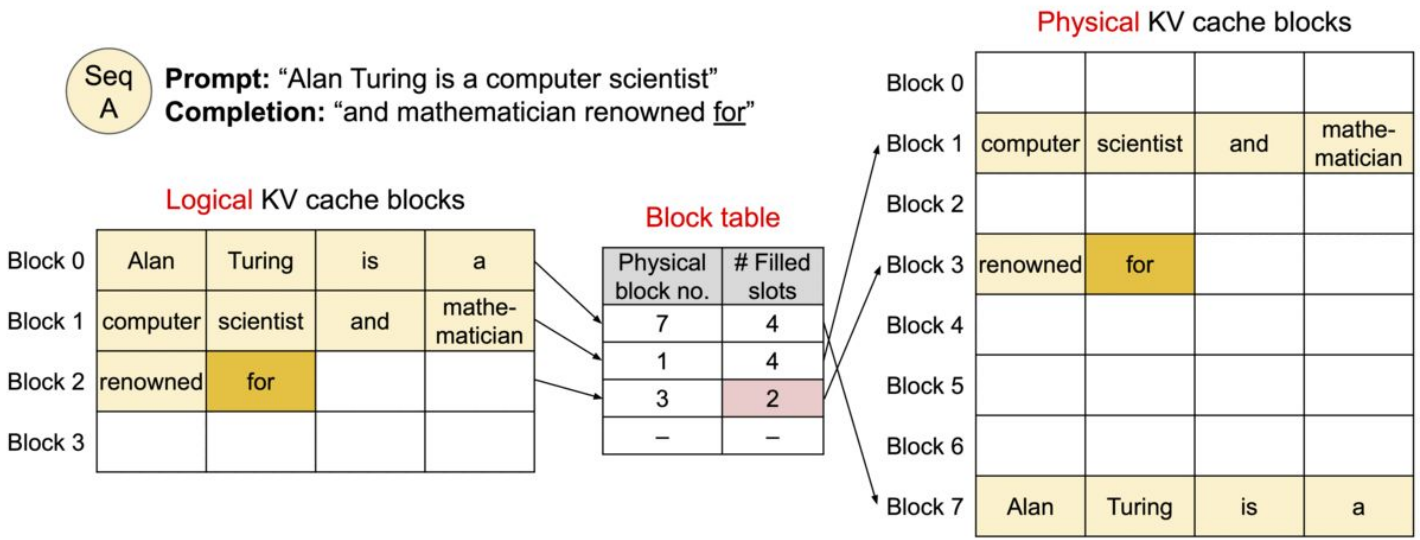
Computing Attention over Paged Blocks



- The math of attention is unchanged – only the memory layout changes

Growing the KV Cache On Demand

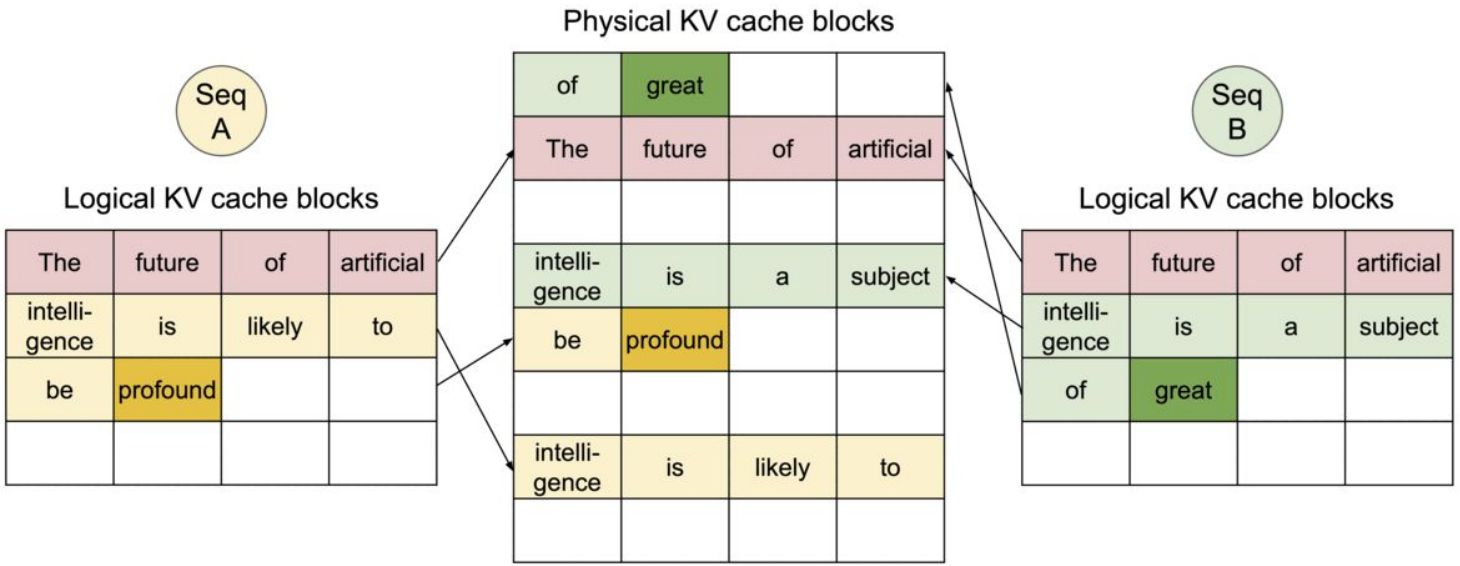
5. Generated 4th token.



- Waste only in the last partially-filled block

Memory Sharing and Copy-on-Write

6. Seq A and B generated 4th token.

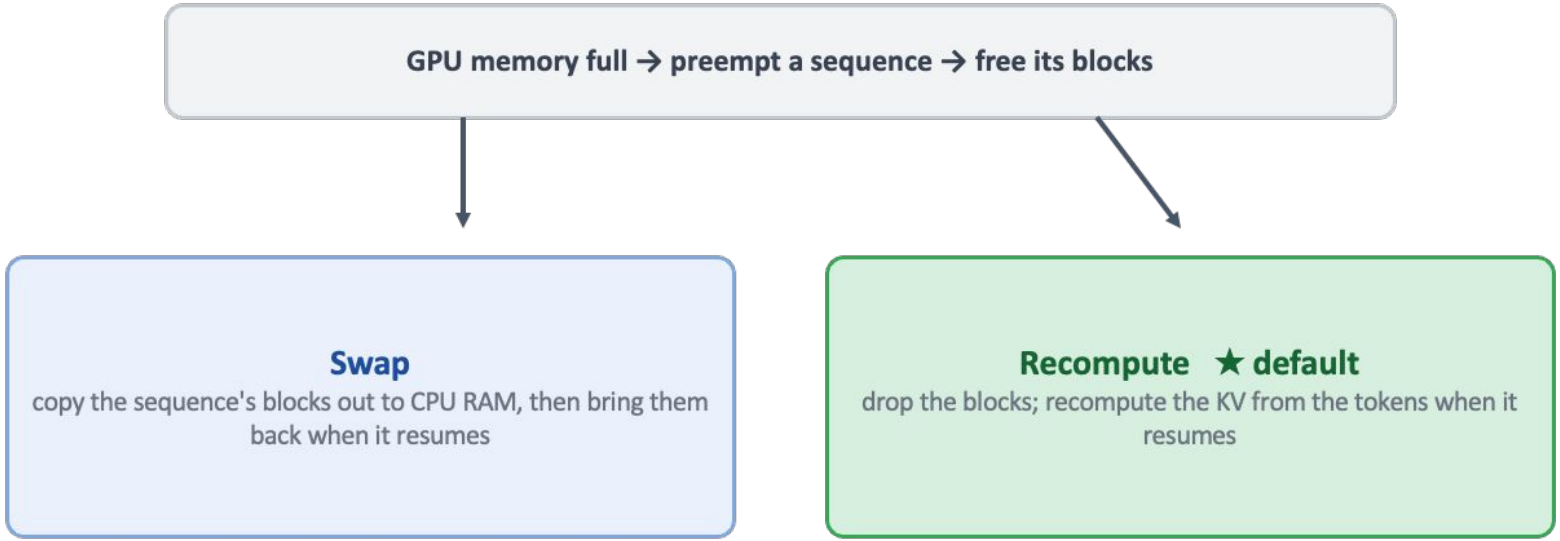


Copy-on-Write: OS vs. vLLM

	OS fork — CoW	vLLM — CoW
Start sharing	parent & child share pages	sequences share KV blocks (ref count)
On write	page fault on the shared page	scheduler sees ref count > 1
Trap	hardware trap → kernel handles	no trap — checked in software
Copy	OS copies the page	vLLM copies the block, remaps table

Same idea — software-orchestrated, not hardware-triggered.

Swapping and Recomputation



Two recovery modes when memory runs out — vLLM defaults to recompute.

Why Custom Kernels

- *Gathering KV from scattered bloks isn't free.*

Existing kernels assume contiguous KV — vLLM's blocks are scattered, so the kernel must gather them via the block table.

3 fused CUDA kernels

reshape + write
store new K, V into the block slots

block read + attention
read blocks via the table and attend, in one pass

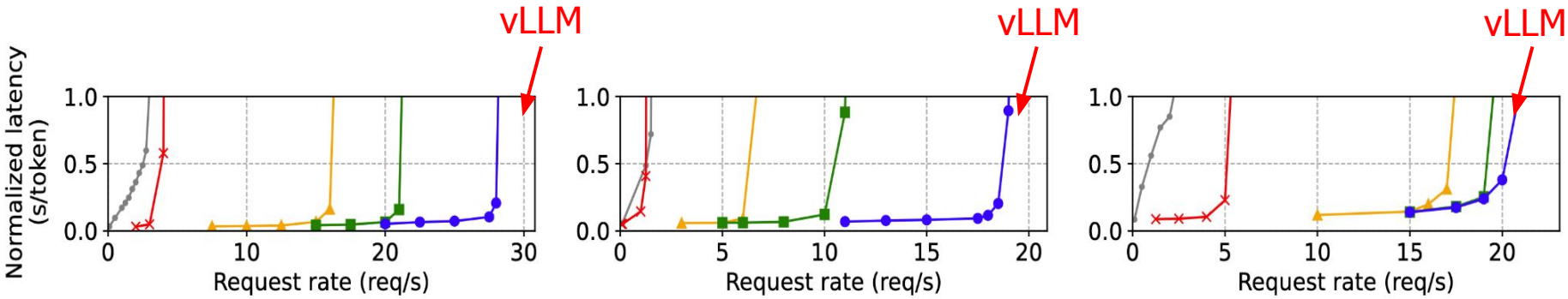
block copy
batch many block copies (copy-on-write) into one launch

Fused kernels keep all that gathering nearly free.

Experimental Setup

- Setup:
  - Models:
    - OPT-13B / 66B / 175B
    - LLaMA-13B
  - Workloads:
    - ShareGPT (long) / Alpaca (short)
  - Hardware: NVIDIA A100
  - Baselines:
    - FasterTransformer, Orca (Oracle / Pow2 / Max)
  - Metric:
    - Normalized latency = end-to-end latency / output length

2-4x Higher Throughput than Prior Systems

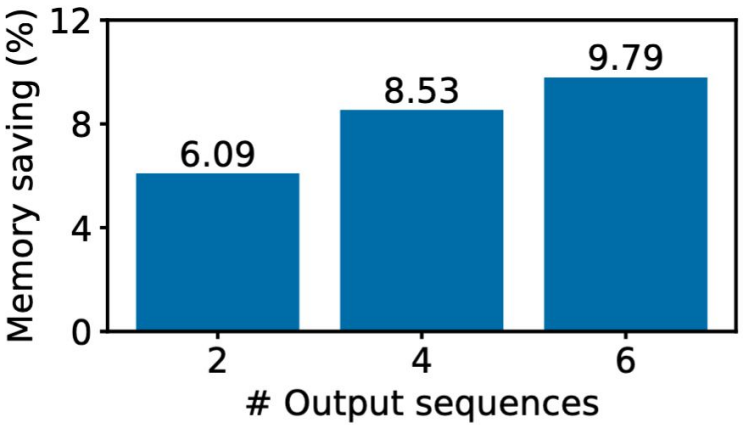


(d) OPT-13B, 1 GPU, Alpaca

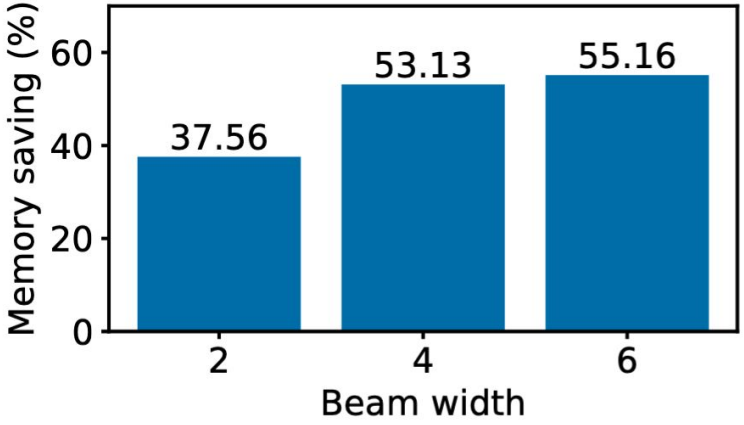
(e) OPT-66B, 4 GPUs, Alpaca

(f) OPT-175B, 8 GPUs, Alpaca

Memory Sharing Drives the Gains

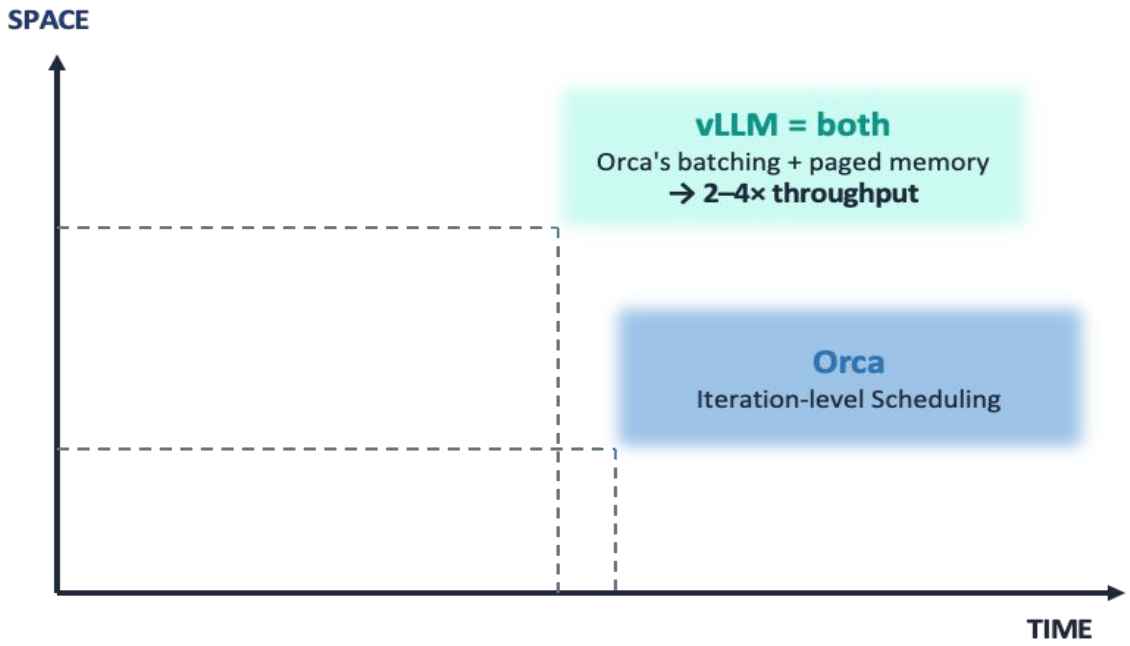


(a) Parallel sampling

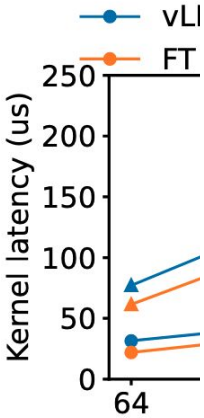


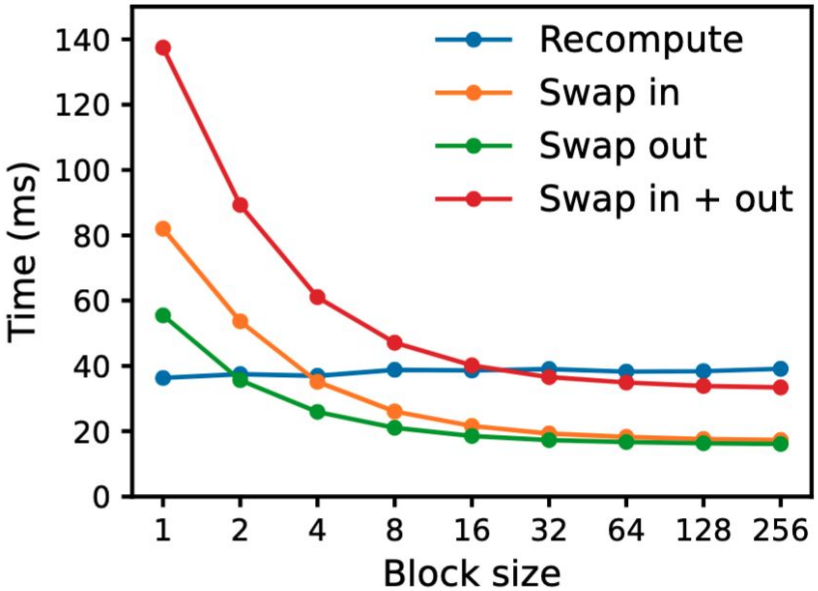
(b) Beam search

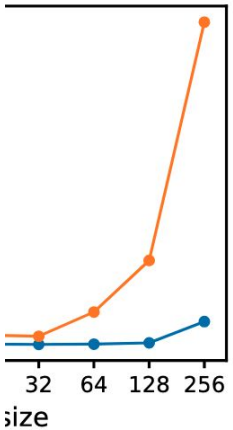
vLLM and Orca: Space vs. Time



Ablations: Costs and Knobs





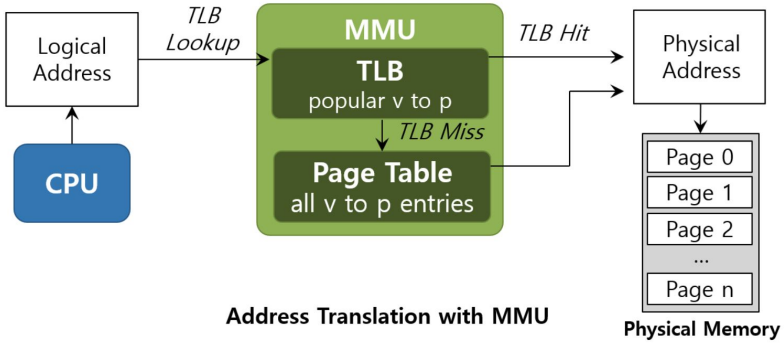


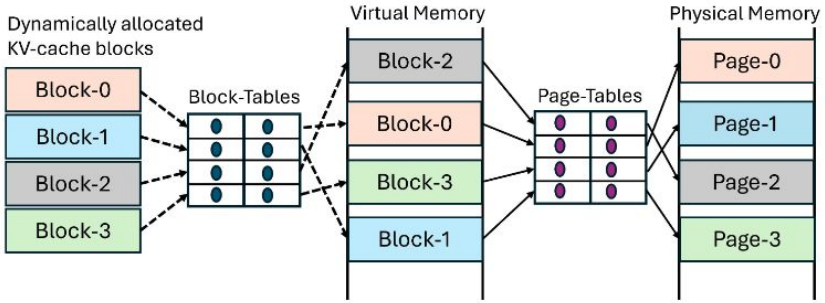
(a) Latency ncy with dif-

ferent block sizes.

PagedAttention as a Software MMU

- Why rewrite the attention kernel at all?





vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention (ASPLOS 25)

Page-table walk done in **hardware**

Block-table walk done in **software**

Why the Block Table Lives in User Space

- NVIDIA does expose hardware paging – so why software?

	GPU hardware paging	vLLM software paging
Protection	enforced (costly)	not needed — one trust domain
Granularity	2 MB page (fixed minimum)	16-token block — tunable per model
Remap cost	CUDA call = kernel round-trip	a pointer write

No protection · finer granularity · no round-trip → keep the mapping in user space.

Limitations and the vAttention(Asplos'25) Critique

hand-tuned block size | kernel rewrite | FCFS head-of-line | no isolation

PagedAttention

block table

non-contiguous KV

custom kernel

"the app manages memory"

vs

vAttention

no block table

contiguous KV + GPU VMM

unmodified kernels

"the system manages memory"

Exokernel question: app vs system · vAttention only ~20% faster · still open